DATA2410 Reliable Transport Protocol (DRTP)

Candidate Number: 370

1.1. Introduction

The DATA2410 Reliable Transport Protocol (DRTP) is a protocol designed to ensure reliable data transmission over a network. Operating on top of UDP, DRTP utilizes the Go-Back-N (GBN) algorithm and a sliding window mechanism to guarantee that data is delivered completely and in the correct order. This approach allows multiple packets to be sent concurrently, leading to more efficient use of network resources. In the event of packet loss or out-of-order delivery, the protocol automatically retransmits the missing packet.

For implementation, a Python-based program was developed to transfer files between two network nodes. The program is launched with arguments specifying the operating mode (client or server), IP address, port number, file path, window size, and test scenario. The client divides the selected file into small packets and transmits them to the server, which reconstructs an identical copy of the original file from the received packets.

The DRTP transmission process is divided into three main phases: connection establishment (SYN-SYN-ACK-ACK), data transfer, and connection termination (FIN-FIN-ACK). These phases are implemented in a manner similar to the connection management of the TCP protocol. The program ensures reliable data delivery by managing the sliding window, handling retransmissions, addressing packet loss, managing timeouts, and controlling out-of-order packets.

The main function of the program, which manages the file transfer logic, is in the application.py file. The application can operate either in client or server mode using command-line arguments. All communication occurs over UDP, while reliability is ensured by the DRTP protocol.

Each DRTP packet carries a fixed-length header preceding the data section. This header contains the sequence number (4 bytes), acknowledgment number (4 bytes), control flags (1 byte; SYN, ACK, FIN), window size (1 byte), and additional fields reserved for alignment or future use. The header provides the essential information required for protocol reliability and control.

Files used in the implementation:

application.py: The main entry point and argument handler.

client.py: Contains client logic (sending packets, window management, retransmission).

server.py: Contains server logic (receiving packets, writing files, generating ACKs).

filename_utils.py: Utility functions for generating unique filenames on the server.

simple-topo.py: Mininet topology and network setup script (for testing only, not run independently).

Command-Line Flags and Their Meanings

Short Flag	Long Flag	Mode	Description
-c	--client	Client	Starts the application in client mode
-s	--server	Server	Starts the application in server mode
-f	--file	Client	Specifies the file to be sent
-i	--ip	Both	IP address to connect/bind to
-p	--port	Both	UDP port to use
-w	--window	Client	Sliding window size for Go-Back-N
-o	--output	Server	Output filename for the received file
-d	--discard	Server	Discards a specific sequence number once (for packet loss testing)

1.2. DRTP Application Architecture and Operation

1.2.1. General Description and Execution Setup

The DRTP protocol provides reliable and ordered file transmission over UDP by adding custom mechanisms on top of its unreliable nature. Typically, the application is deployed in a Linux environment, where a topology script (such as sudo python3 simple-topo.py) is executed to set up the network. Mininet is used for network emulation, and both the client (h1) and server (h2) are launched in separate xterm terminals. It is important to start the server before running the client. The application can be configured using command line arguments to specify the mode (client or server), target IP address, port, file for transfer, window size, and optional parameters like packet loss simulation. Output from each session can be saved using tools like tee for future reference.

Connection Setup (Three-Way Handshake)

Prior to any data exchange, a three-way handshake takes place between the client and server, involving SYN, SYN-ACK, and ACK messages. This process ensures that both ends are ready to communicate and that all connection parameters are properly initialized. If the connection attempt is unsuccessful, the client notifies the user of the error.

File Transfer, Flow Control, and Reliability

After the connection is established, the client reads the selected file, divides it into 992-byte segments (with each packet carrying an 8-byte header), and transmits the packets using a Go-Back-N sliding window protocol. The client can send several packets up to the window limit before waiting for acknowledgments (ACKs) from the server. If an ACK is not received within the expected time (indicating packet loss or timeout), the client resends all unacknowledged packets in the current window.

On the server side, only the expected in-order packets are written to the output file and acknowledged. If a packet arrives out of order, it is discarded (not buffered), and the server

continues to acknowledge the last correctly received in-order packet. This behavior is specific to the Go-Back-N protocol and ensures reliable, ordered delivery of data.

Retransmission and Packet Loss Simulation (Testing)

To verify the robustness of the protocol's reliability features, the server can be started with a specific flag (such as -d) to deliberately drop a particular packet once. This emulates packet loss and causes the client's retransmission logic to activate. Such targeted tests help demonstrate the protocol's ability to recover from lost or dropped packets.

Connection Teardown (Session Termination)

Once the file transfer is complete and all packets have been acknowledged, the client initiates session termination by sending a FIN packet. The server answers with a FIN-ACK message. This orderly process allows both sides to close the connection gracefully and securely.

2.1. Test Scenarios

2.1.1. Effect of Window Size (-w) on Throughput

In each experiment, the file transfer was initiated using application.py from **h1 (client)**, transmitting a photo file (iceland-safiqul.jpg) to **h2 (server)**. The window size parameter (-w) was varied sequentially as 3, 5, 10, 15, 20, and 25. For every configuration, the outputs from both **h1** and **h2** were recorded separately. After each file transfer, the original and received files were compared using the md5sum command to ensure data integrity.

Window Size (-w)	Throughput (Mbps)	md5sum Check
3	0.22	Successful
5	0.37	Successful
10	0.69	Successful
15	0.93	Successful
20	0.91	Successful
25	0.97	Successful

Interpretation of Results

Throughput increases substantially as the window size increases. This is because, in the Go-Back-N protocol, a larger window size allows more packets to be "in flight" simultaneously, making more efficient use of the available bandwidth and reducing idle time due to acknowledgments.

After a window size of 20 (e.g., at 25), further increases in window size no longer yield significant improvements in throughput. At this point, the network's physical constraints—such as round-trip time (RTT), packet loss, and hardware limitations—become the primary limiting factors.

All tests successfully completed the file transfer without data loss or corruption. This was confirmed by md5sum checks (see figure below), with a 100% match between the original and received files.

Details and Observations from the Transfer

h1 (Client) and h2 (Server) Output Analysis

For each window size, the outputs from **h1** and **h2** were carefully reviewed (for example, see "h1-w3.txt" and "h2-w3.txt" files as references).

Connection setup followed a standard three-way handshake (SYN, SYN-ACK, ACK).

During data transfer: With a small window size, the sender had to wait for acknowledgments more frequently, causing interruptions in data flow and more timeouts. As the window size increased, the pipeline stayed fuller, resulting in higher throughput.

During the connection teardown, simulated loss of FIN-ACK packets occasionally occurred. In these scenarios, the client (h1) retransmitted the FIN packet, demonstrating the protocol's reliability.

Packet Ordering and Timeouts

The **h2** outputs frequently reported "out-of-order" packets. This is an expected characteristic of the Go-Back-N protocol: the receiver only acknowledges the last correctly received in-order packet and discards out-of-order arrivals. In all cases, the file transfer was completed successfully, and the throughput was measured.

Protocol Reliability

After every transfer, the md5sum values of the received files and the original file were compared, resulting in a 100% match. This demonstrates that the protocol is both reliable and maintains data integrity throughout the transfer process. Even with large window sizes, the Go-Back-N protocol's timeout and retransmission mechanisms operated correctly.

General Evaluation and Limitations

In these tests, RTT was kept constant at 100ms. Additional scenarios with varying RTT and packet loss rates are discussed in subsequent sections. While increasing the window size leads to significant throughput improvements, network physical limits (such as bandwidth, RTT, and loss rate) ultimately restrict further gains.

The protocol continued to deliver data reliably, even in the presence of packet loss or out-of-order delivery, due to the Go-Back-N mechanism.

Limitation: All tests were conducted using a single file (iceland-safiqul.jpg) and a specific Mininet topology (simple-topo.py). Further tests with different file sizes and topologies are recommended for broader validation.

2.1.2. Testing different RTT values

2.1.2.1. Value With RTT = 50 ms

Window Size (-w)	Throughput (Mbps)	md5sum Check
3	0.45	Successful
5	0.69	Successful
10	1.21	Successful
15	1.81	Successful
20	1.77	Successful
25	1.78	Successful

Analysis and Interpretation

When the RTT is set to 50 ms, increasing the sliding window size (w) results in a substantial rise in throughput. Specifically, the throughput increases significantly as the window size is raised from 3 to 15. However, after w = 15, further increasing the window size yields almost no additional gain in throughput, as the values plateau and remain very close to each other for w = 15, 20, and 25.

Significance of the Results

Initial Growth: At lower window sizes, the sender is constrained in the number of packets that can be sent concurrently before waiting for acknowledgements (ACKs). By increasing the window size, more packets can be kept "in flight," resulting in improved network utilization and higher throughput.

Saturation Point: From w = 3 to w = 15, the throughput increases nearly linearly (0.45 → 0.69 → 1.21 → 1.81 Mbps). However, for window sizes beyond w = 15 (i.e., w = 20 and w = 25), the throughput remains essentially unchanged (1.77, 1.78 Mbps).

Explanation: This plateau effect occurs because the sender reaches the network's bandwidth-delay product (BDP) limit. At this point, the maximum amount of data that can be "in flight" in the network has been reached, and any further increase in the window size does not lead to improved throughput.

Network Capacity Limit: The BDP is determined by the link bandwidth and the round-trip time. Once the sender fully utilizes this capacity, adding more outstanding packets only increases the number of packets buffered in the network but does not improve the rate at which data is delivered.

Data Integrity: In every experiment, md5sum checks confirmed that the transferred file matched the original perfectly. This demonstrates that the DRTP protocol provides reliable and lossless data transfer, regardless of window size changes or retransmissions.

Practical Implication: The results highlight that optimal throughput is achieved when the window size is tuned to match the network's BDP. Increasing the window size beyond this point does not confer any additional performance benefit and may only add unnecessary buffering or resource usage on the sender's side.

2.1.2.2. Value with RTT = 200 ms

Window Size (-w)	Throughput (Mbps)	md5sum Check
3	0.11	Successful
5	0.19	Successful
10	0.36	Successful
15	0.53	Successful
20	0.53	Successful
25	0.54	Successful

Analysis and Interpretation

Data Integrity:
Across all tests, the transferred files' md5sum values matched those of the original files. This demonstrates that the DRTP protocol implementation reliably preserved data integrity even under high-latency conditions.

Throughput Behavior:
Throughput increases steadily with larger window sizes:

- 0.11 Mbps at w=3
- 0.19 Mbps at w=5
- 0.36 Mbps at w=10
- Plateauing at approximately 0.53–0.54 Mbps for w=15, 20, and 25

Interpretation of Results:
At lower window sizes, the sender cannot fully utilize the network's bandwidth because it must wait for acknowledgments before sending more packets. As the window size increases, the sender is allowed to transmit more data before requiring an acknowledgment, resulting in higher throughput. However, after a certain point (around w=15 in these experiments), increasing the window size further yields no significant throughput improvement, the throughput plateaus.

Explanation:
This behavior indicates that the limiting factor has shifted from the sender's window size to the network's bandwidth-delay product (BDP). Once the sender's window is large enough to keep the pipeline full—meaning the sender always has enough unacknowledged packets in flight to saturate the link—the maximum throughput is reached. Any additional increase in the window size does not further improve performance, as the protocol is already operating at the network's capacity.

Impact of RTT:
With a high round-trip time (RTT) of 200 ms, a small window size results in significant idle time as the sender waits for acknowledgments, severely limiting throughput. When the window size is increased, the sender spends less time idle, thus throughput increases until the maximum possible value is reached, determined by the bandwidth-delay product.

Theoretical Context:
These results align with the theoretical expectations for Go-Back-N protocols. Throughput is limited by the minimum of the sender's window size and the available bandwidth-delay

product of the network. The plateau observed from w=15 onwards represents the point at which the window is sufficiently large for the sender to continuously transmit data, fully utilizing the available network bandwidth even under high-delay conditions.

2.1.2.3. Evaluation Of 3 Different RTT Tests

The experiments investigated how varying the Round-Trip Time (RTT)—specifically, 50 ms, 100 ms, and 200 ms—impacts file transfer throughput using different window sizes (-w). For each RTT setting, throughput values were measured for window sizes of 3, 5, 10, 15, 20, and 25. The results are summarized in the following table:

RTT (ms)	w=3	w=5	w=10	w=15	w=20	w=25
50	0.45	0.69	1.21	1.81	1.77	1.78
100	0.22	0.37	0.69	0.93	0.91	0.97
200	0.11	0.19	0.36	0.53	0.53	0.54

Analysis and Discussion

As RTT increases, throughput drops significantly, especially for smaller window sizes. This behavior is expected in sliding window protocols like Go-Back-N: with a high RTT, the sender must wait longer for acknowledgments before sending new packets, which reduces overall efficiency. When RTT is low (50 ms), increasing the window size leads to rapid throughput gains, allowing the protocol to make fuller use of the available bandwidth.

At an RTT of 100 ms, throughput grows more gradually with window size but still shows noticeable improvement up to a point. In the highest RTT scenario (200 ms), throughput is significantly lower, and after a window size of 15, further increases in the window size yield minimal additional benefit. This plateau occurs because, as RTT rises, the delay-bandwidth product (BDP) increases, and the protocol becomes more limited by network latency rather than window size.

These results are consistent with bandwidth-delay product theory: higher RTT requires a larger window to maximize throughput, but after a certain point, other factors such as protocol overhead and network congestion prevent further gains.

In summary, lower RTTs enable higher throughput, especially when the window size is increased. As RTT rises, the impact of increasing the window size diminishes, and the maximum achievable throughput decreases, demonstrating the fundamental limitations imposed by high-latency networks.

2.1.3. Testing with different -d (--discard) values

2.1.3.1. With value -d 1

Window Size (-w)	Throughput (Mbps)	md5sum Check
3	0.22	Successful
5	0.37	Successful
10	0.70	Successful
15	0.94	Successful
20	0.92	Successful
25	0.94	Successful

Analysis and Discussion

In these tests, the server is configured to deliberately drop a specific packet (for example, the first packet). When the client does not receive an ACK for this packet within the retransmission timeout, it retransmits the lost and all subsequent packets in accordance with the Go-Back-N protocol. The server then successfully receives the retransmitted packet and continues with the rest of the file transfer.

Functionality of Go-Back-N Mechanism: The results demonstrate that the Go-Back-N mechanism implemented by the protocol reliably handles both lost and intentionally dropped packets. Upon a timeout, the client retransmits all unacknowledged packets, and the server, upon receiving the missing packet, acknowledges and delivers data in order. This mechanism ensures reliable and in-order delivery over an unreliable network.

Throughput Increase and Saturation: As the window size increases, throughput also increases, indicating improved utilization of the network capacity. However, after a window size of 15, throughput gains diminish, and values remain very similar for window sizes of 15, 20, and 25. This plateau indicates that the protocol has reached the network's bandwidth-delay product limit, and further increasing the window size does not yield additional throughput improvement.

No Data Loss or Corruption: In all test cases, the transferred file was received without any data loss or corruption, as verified by matching md5sum checksums for the original and received files. This confirms that the protocol's retransmission mechanism does not introduce errors, and file integrity is preserved.

Error Handling: No persistent errors, connection losses, or interruptions were observed in the logs. The protocol effectively recovers from the intentional packet drop and successfully completes the transfer in all scenarios.

Conclusion

The DRTP protocol, built on top of UDP and employing the Go-Back-N algorithm, successfully achieves reliable, in-order, and error-free file transfer. Even when intentional packet loss is introduced, the protocol recovers without user intervention, and the file is delivered intact. Increasing the window size improves throughput up to a certain point, after which network constraints prevent further improvement. The experiment validates the protocol's robustness and effectiveness under packet loss conditions.

2.1.3.2. With value -d 1 and -d 5

Window Size (-w)	**Throughput (-d 1)** / Mbps	**Throughput (-d 5)**	**Md5sum Check**
3	0.22	0.22	Successful
5	0.37	0.37	Successful
10	0.70	0.65	Successful
15	0.94	0.97	Successful
20	0.92	0.93	Successful
25	0.94	0.95	Successful

Analysis and Discussion

In these experiments, the server was configured to deliberately drop a specific data packet to simulate packet loss. In the first test (-d 1), the first packet was dropped; in the second test (-d 5), the fifth packet was dropped. In both scenarios, the client retransmitted the lost and all subsequent packets after a timeout, as required by the Go-Back-N protocol. Upon receiving the missing packet, the server resumed in-order delivery, and the file transfer was completed successfully.

Functionality of the Go-Back-N Mechanism:

The results confirm that the Go-Back-N mechanism handles both early and mid-transfer packet loss reliably. When a timeout occurs due to a missing acknowledgment, the client retransmits all unacknowledged packets. Once the server receives the missing packet, it accepts the subsequent packets and completes the file delivery in order. This behavior demonstrates the protocol's ability to ensure reliable and in-order transmission over an unreliable network.

Meaning of Changing the -d Value:

The -d flag determines which packet will be intentionally dropped during transmission. Using -d 1 causes a packet loss at the very beginning of the transfer, while -d 5 introduces the loss slightly later. Despite this difference, the protocol's behavior remains consistent across both cases. Minor differences in throughput may be due to timing variations or slight differences in how the retransmissions affect the overall transfer duration, but they do not indicate any protocol malfunction.

Throughput Comparison:

Throughput increases as the window size increases in both sets of tests, showing better utilization of the network. After a window size of 15, the throughput levels off, suggesting that the bandwidth-delay product limit of the network has been reached. Interestingly, the throughput values for -d 5 at window sizes 15, 20, and 25 are slightly higher than those for -d 1, possibly due to favorable timing or retransmission overlap. These differences are small and do not indicate any substantial variation in protocol behavior.

Data Integrity and Error Handling:

In all test cases, the md5sum checksum verification confirmed that the transferred files were intact, without any data corruption. The protocol's retransmission logic did not introduce any errors. Additionally, there were no persistent errors, connection failures, or interruptions during the transfer process.

Conclusion

The DRTP protocol, built on top of UDP and implementing the Go-Back-N algorithm, demonstrates robust and reliable performance in the face of intentional packet loss, whether early in the transfer (-d 1) or slightly later (-d 5). The protocol correctly handles retransmissions, ensures in-order delivery, and preserves file integrity. Increasing the window size improves throughput until it reaches a saturation point determined by the network conditions. These tests validate the protocol's effectiveness, reliability, and resilience in handling packet loss scenarios.

2.1.4. Loss tests

2.1.4.1. Test Values with 2% Random Packet Loss and 100ms Delay

Window Size (-w)	Throughput (Loss 2%)	md5sum Check
3	0.22 Mbps	Successful
5	0.37 Mbps	Successful
10	0.70 Mbps	Successful
15	1.04 Mbps	Successful
20	0.95 Mbps	Successful
25	0.99 Mbps	Successful

Analysis and Discussion

In this set of experiments, 2% random packet loss and a constant 100ms delay were introduced on the network link using traffic control (tc netem). Unlike deliberate drops in specific packets (like -d 1 or -d 5), the 2% loss scenario simulates real-world network behavior with unpredictable losses scattered throughout the transmission.

Functionality of the Go-Back-N Mechanism: The results show that the Go-Back-N mechanism effectively manages random packet loss. When a packet is lost or its acknowledgment is not received, the client initiates a timeout and retransmits all unacknowledged packets. This ensures that the server eventually receives the missing packet and resumes in-order delivery. Despite randomness in loss location, the protocol consistently completes the transmission successfully.

Impact of Random Loss: The 2% loss rate introduces more frequent retransmission events compared to single deliberate drops. This causes increased network congestion and redundant traffic, especially for smaller window sizes. However, the protocol shows no instability or breakdown even under sustained loss.

Throughput Behavior: As the window size increases, throughput improves significantly, especially from window size 3 to 15. Beyond -w 15, the throughput gains plateau, indicating that the bandwidth-delay product (BDP) limit of the network has been reached. Minor fluctuations at w=20 and w=25 may be due to variations in the timing of losses and retransmissions, but they do not signify protocol failure.

Data Integrity and Error Handling: All files were transferred successfully without corruption, as verified by consistent md5sum hashes. This confirms that the retransmission strategy of the protocol preserves data integrity. Additionally, there were no persistent errors or unexpected termination events throughout the tests. Connection closure issues (e.g., missing FIN-ACK) were handled gracefully.

Conclusion

The DRTP protocol demonstrates strong reliability and resilience under realistic network conditions with 2% random packet loss. Built on UDP and enhanced by the Go-Back-N algorithm, the protocol ensures in-order delivery and recovers from unpredictable packet drops without user intervention. Increasing the window size enhances performance up to a saturation point, after which network constraints dominate. These results validate the protocol’s robustness and effectiveness in real-world, lossy environments.

2.1.4.2. Test Values with 5% Random Packet Loss and 100ms Delay

Window Size (-w)	Throughput (Loss 2%)	md5sum Check
3	0.22 Mbps	Successful
5	0.36 Mbps	Successful
10	0.72 Mbps	Successful
15	1.02 Mbps	Successful
20	1.00 Mbps	Successful
25	1.00 Mbps	Successful

Analysis and Discussion

Client Side (h1) Observations

Connection Setup: In all cases, the connection was successfully established through the standard SYN, SYN-ACK, ACK handshake.

Retransmission Behavior: Multiple retransmissions of the same packet sequences were observed, especially at lower window sizes. This behavior is consistent with the Go-Back-N protocol, where a single lost packet causes the retransmission of the entire window. For instance, packet sequences such as seq=1–5 or seq=10–15 were seen repeating up to 3–6 times in some tests.

Repeated Patterns: High levels of repetition, particularly at -w 25, were recorded. Some packets were retransmitted excessively (6–7 times), suggesting timeout-triggered resend loops due to delayed or lost ACKs.

Connection Termination: Frequent log entries such as "FIN-ACK not received, retrying..." indicate packet loss during the termination phase. Despite this, sessions eventually timed out gracefully.

Server Side (h2) Observations

Out-of-Order Reception: Log entries frequently reported "out-of-order packet seq=X, expected=Y", confirming that subsequent packets were discarded until the missing one arrived. This matches Go-Back-N behavior, which enforces in-order delivery.

Recovery After Missing Packet: Once the missing packet was received, all subsequent buffered packets were processed correctly, restoring order and allowing transmission to proceed.

Timeout Warnings: The message Timeout while receiving data was consistently observed at the end of each transfer, indicating that the server waited for additional packets before eventually closing the session.

Data Integrity: In all six window size scenarios, the transferred files produced identical md5sum values, confirming successful and complete delivery without corruption. No session crashed or failed despite persistent loss and retransmission events.

Performance Trends: Throughput improved significantly with increasing window size, from 0.22 Mbps (-w 3) to over 1 Mbps (-w 15–25). Performance gains began to plateau after -w 15, suggesting that the network's bandwidth-delay product was reached. Despite increased packet loss, throughput remained relatively stable at high window sizes, confirming the efficiency of the protocol in absorbing random losses.

The DRTP protocol, based on UDP and enhanced by Go-Back-N retransmission, demonstrated strong reliability and robustness under a 5% random packet loss scenario. Regardless of loss position or frequency, the protocol successfully ensured in-order, complete, and error-free data delivery. Large window sizes significantly improved throughput, while retransmission logic ensured resilience and correctness throughout the transmission process.

2.1.4.3. Test Values with 50% Random Packet Loss and 100ms Delay

Window Size (w)	Throughput (Mbps)	File Integrity (MD5 Match)	FIN-ACK Failures (Retries)	Out-of-Order Bursts (h2 logs)
3	0.22	Successful	5	Many duplicates of 1839–1841
5	0.36	Successful	5	High repetition of 1837–1841
10	0.69	Successful	5	Repetitions remain, more balanced behavior
15	0.98	Successful	5	~13 repeated logs of seq=0, bursts still visible
20	0.87	Successful	5	Bursty logging of 1838–1841 and seq=0
25	1.02	Successful	0	Long bursts continue, but recovery is more consistent

Analysis and Discussion

Under an extremely adverse network condition of 50% random packet loss, the DRTP protocol based on Go-Back-N reliably completed all transmissions without data corruption. Despite the high loss rate, all transferred files passed md5sum verification, confirming that the retransmission logic correctly ensured delivery integrity. Throughput values increased with window size, peaking at 1.02 Mbps for w=25, while the lowest throughput occurred at w=3 (0.22 Mbps), highlighting how larger windows mitigate the impact of loss by increasing pipeline utilization.

System behavior under these conditions reflected typical Go-Back-N dynamics. Frequent retransmissions occurred due to lost or delayed ACKs. In the logs, sequence numbers such as 1837–1841 and packet 0 were repeatedly sent and logged as "out-of-order" at the receiver side. This indicates that packets arriving before the expected sequence number were discarded as per protocol. The client (h1) attempted to close the connection five times in all cases except for w=25, where the FIN-ACK was finally received—suggesting that higher window sizes may improve control packet reliability as well.

The server (h2) exhibited intense out-of-order logging behavior, especially with larger windows. The logs show that the same packets were reported as out-of-order repeatedly, potentially due to duplicated arrivals or lack of deduplication in the log system. Despite this, all transfers terminated with complete data delivery. This confirms that while the protocol experiences visible strain under high loss, it maintains correctness and robustness in both flow and reliability.

2.1.4.4.. Window Size vs. Throughput Under 2%, 5%, and 50% Packet Loss Conditions

Window Size (w)	2% Loss	5% Loss	@50% Loss
3	0.22	0.22	0.22
5	0.36	0.36	0.36
10	0.69	0.72	0.69
15	0.98	1.02	0.98
20	0.87	1.00	0.87
25	1.02	1.00	1.02

2.1.4.5.. Analysis and Discussion

As packet loss increases from 2% to 50%, the throughput of the DRTP protocol shows a clear sensitivity to network reliability, particularly at larger window sizes. At small window sizes (w=3 and w=5), throughput remains nearly identical across all loss levels, indicating that retransmission overhead dominates performance regardless of the loss rate. As the window size increases, the throughput improves under all loss conditions, but the rate of improvement slows under higher loss. At 2% and 5% loss, throughput peaks around w=15–20, while at 50% loss, a slightly delayed peak occurs at w=25. This suggests that the protocol compensates for increased loss with a larger sliding window, but the efficiency gains diminish as retransmissions become more frequent.

3.1. Explain what happens when a fin-ack packet is lost.

When a FIN-ACK packet is lost, the expected behavior in a Go-Back-N-based protocol is for the client to retransmit the FIN packet until an acknowledgment is received or a maximum retry limit is reached. Experimental results confirm this behavior: for window sizes between w=3 and w=20, the client retransmitted the FIN packet five times without receiving a FIN-ACK and ultimately closed the connection with a timeout message. This indicates that the retransmission and timeout mechanisms functioned as designed. At w=25, the FIN-ACK was successfully received, and the connection closed normally. This suggests that larger window sizes may increase the likelihood of control packets being successfully delivered, improving the reliability of the connection termination process. Overall, the observed behavior aligns with protocol expectations and confirms that the implementation handled FIN-ACK loss correctly.

References

1. Argparse Tutorial. Available at: https://docs.python.org/3/howto/argparse.html
2. Individual Assignment: DATA2410 Reliable Transport Protocol (DRTP). Available at: https://github.com/safiqul/DRTP-v25
3. Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th Edition, Global Edition). Pearson Education.
4. Mininet Network Emulator. Available at: http://mininet.org/
5. Safiqul Islam (2025), DATA2410 Lab Manuals and Github Resources. Available at: https://github.com/safiqul/2410
6. Struct — Interpret bytes as packed binary. Available at: https://docs.python.org/3/library/struct.html
7. W3school Python Tutorial. Available at: https://www.w3schools.com/python/